



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Fall, Year: 2025), B.Sc. in CSE (Day)

LAB REPORT NO : 04

Course Title: Operating System Lab

Course Code: CSE-402 Section: 231-D2

Student Details

Name		ID
1.	Promod Chandra Das	231002005

Lab Date : 18-12-25

Submission Date: 25-12-25

Course Teacher's Name : Md. Shoab Alam

[For Teachers use only: **Don't Write Anything inside this box**]

<u>Lab Report Status</u>	
Marks:	Signature:
Comments:	Date:

1. TITLE OF THE LAB EXPERIMENT :

Comparison Analysis of Best Fit, First Fit, and Worst Fit Memory Allocation Techniques

2. OBJECTIVES :

- To understand dynamic memory allocation strategies used in operating systems.
- To implement First Fit, Best Fit, and Worst Fit algorithms using shell scripting.
- To compare these algorithms based on memory utilization, fragmentation, and allocation efficiency.
- To analyze which strategy performs better under different memory request patterns.

3. PROCEDURE :

- Define a set of memory blocks with fixed sizes.
- Define a set of process memory requests.

Implement:

- **First Fit:** Allocate the first block large enough.
- **Best Fit:** Allocate the smallest block sufficient for the process.
- **Worst Fit:** Allocate the largest available block.
- Display allocation results and remaining memory.
- Compare performance and fragmentation behavior.

4. IMPLEMENTATION :

```
#!/bin/bash

# Memory blocks and process sizes
blocks=(100 500 200 300 600)
processes=(212 417 112 426)

n=${#blocks[@]}
m=${#processes[@]}

echo "Memory Blocks: ${blocks[@]}"
echo "Processes: ${processes[@]}"
echo

# ----- First Fit -----
echo "FIRST FIT"
ff_blocks=("${blocks[@]}")

for ((i=0; i<m; i++)); do
    allocated=0
    for ((j=0; j<n; j++)); do
        if [ ${ff_blocks[j]} -ge ${processes[i]} ]; then
            echo "Process ${processes[i]} allocated to block $j"
            ff_blocks[j]=$((ff_blocks[j] - processes[i]))
            allocated=1
            break
        fi
    done
    [ $allocated -eq 0 ] && echo "Process ${processes[i]} not allocated"
done
echo

# ----- Best Fit -----
echo "BEST FIT"
bf_blocks=("${blocks[@]}")

for ((i=0; i<m; i++)); do
    best=-1
    for ((j=0; j<n; j++)); do
        if [ ${bf_blocks[j]} -ge ${processes[i]} ]; then
```

```

    if [ $best -eq -1 ] || [ ${bf_blocks[j]} -lt ${bf_blocks[best]} ]; then
        best=$j
    fi
fi
done
if [ $best -ne -1 ]; then
    echo "Process ${processes[i]} allocated to block $best"
    bf_blocks[best]=$((bf_blocks[best] - processes[i]))
else
    echo "Process ${processes[i]} not allocated"
fi
done
echo

# ----- Worst Fit -----
echo "WORST FIT"
wf_blocks=("${blocks[@]}")

for ((i=0; i<m; i++)); do
    worst=-1
    for ((j=0; j<n; j++)); do
        if [ ${wf_blocks[j]} -ge ${processes[i]} ]; then
            if [ $worst -eq -1 ] || [ ${wf_blocks[j]} -gt ${wf_blocks[worst]} ]; then
                worst=$j
            fi
        fi
    done
    if [ $worst -ne -1 ]; then
        echo "Process ${processes[i]} allocated to block $worst"
        wf_blocks[worst]=$((wf_blocks[worst] - processes[i]))
    else
        echo "Process ${processes[i]} not allocated"
    fi
done

```

OutPut

```
promod@Ubuntu24:~$ touch Labreport04.sh
promod@Ubuntu24:~$ chmod +x Labreport04.sh
promod@Ubuntu24:~$ ./Labreport04.sh
Memory Blocks: 100 500 200 300 600
Processes: 212 417 112 426

FIRST FIT
Process 212 allocated to block 1
Process 417 allocated to block 4
Process 112 allocated to block 1
Process 426 not allocated

BEST FIT
Process 212 allocated to block 3
Process 417 allocated to block 1
Process 112 allocated to block 2
Process 426 allocated to block 4

WORST FIT
Process 212 allocated to block 4
```

Figure: 01

```
Processes: 212 417 112 426

FIRST FIT
Process 212 allocated to block 1
Process 417 allocated to block 4
Process 112 allocated to block 1
Process 426 not allocated

BEST FIT
Process 212 allocated to block 3
Process 417 allocated to block 1
Process 112 allocated to block 2
Process 426 allocated to block 4

WORST FIT
Process 212 allocated to block 4
Process 417 allocated to block 1
Process 112 allocated to block 4
Process 426 not allocated
promod@Ubuntu24:~$
```

Figure: 02

5. ANALYSIS AND DISCUSSION

✓ First Fit

- Fastest execution.
- Causes external fragmentation early.
- Leaves scattered small holes.
- Efficient for speed, inefficient for long-term memory use.

✓ Best Fit

- Minimizes leftover memory per allocation.
- **High external fragmentation** due to many tiny unused blocks.
- Slower than First Fit due to full search.

✓ Worst Fit

- Leaves large remaining blocks.
- Attempts to avoid fragmentation but **often wastes memory**.
- Performs poorly when large requests arrive later.

Algorithm	Speed	Fragmentation	Memory Utilization
First Fit	Fast	Medium	Medium
Best Fit	Slow	High	Low
Worst Fit	Medium	Low (initially)	Poor

1. SUMMARY

- **First Fit** is best when speed matters.
- **Best Fit** looks smart but backfires due to fragmentation.
- **Worst Fit** sounds logical but wastes memory in practice.
- No algorithm is universally optimal — workload matters.
- Modern systems avoid these problems using **paging and segmentation**, not raw contiguous allocation.

